Prog-Py

FS2026 – Prof. Selina Rea Malacarne

Autoren: Mirko Ratti

https://github.com/P4ntomime/TeXFoSaTemplate

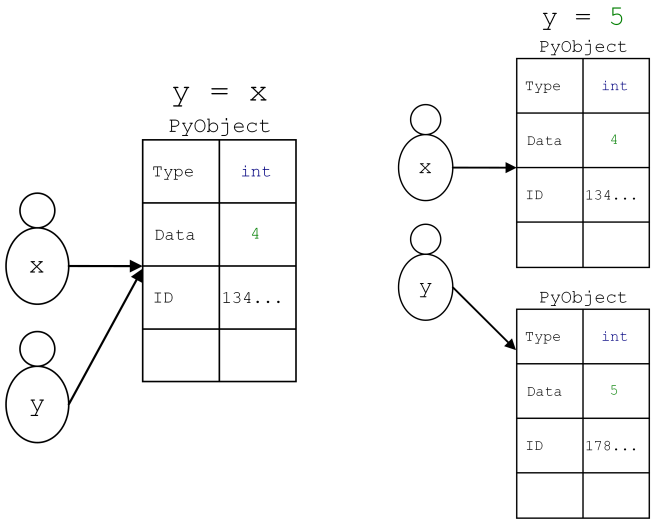
V1.0.20260306

1 Python Overview

- **Origins:** Developed by Guido van Rossum with a focus on simplicity and power.
- **Standard Library:** Known for the "batteries included"philosophy, meaning it comes with an extensive built-in library.
- **Ecosystem:** For engineering tasks, Python replaces Matlab toolboxes with specialized packages like **NumPy** (numerical data), **pandas** (data processing), and **matplotlib** (visualization).

2 Variables and Objects

In Python, everything is an object. Variables are not containers but **references** (pointers) bound to objects.



2.1 Built-in Data Types

Category	Data Type	Mutability	False Value
None	`NoneType`	Immutable	`None`
Numeric	`int`	Immutable	`0`
	`float`	Immutable	`0.0`
	`bool`	Immutable	`False`
	`complex`	Immutable	`0 + 0j`
Sequence	`str`	Immutable	`"`
	`list`	Mutable	`[]`
	`tuple`	Immutable	`()`
	`bytes`	Immutable	`b"`
	`bytearray`	Mutable	`bytearray(b")`
Sets	`set`	Mutable	`set()`
	`frozenset`	Immutable	`frozenset()`
Associative	`dict`	Mutable	`{}`

2.2 Mutability

- **Immutable:** Objects that cannot be modified after creation (e.g., `int`, `float`, `str`, `tuple`). Changing the value of an immutable variable actually creates a new object in memory.
- **Mutable:** Objects that can be modified in-place (e.g., `list`, `set`, `dict`).

The memory address of an object, can be visuasalized using the `id()` function.
It returns a unique integer for the given object.

3 Namespaces and Scope

A namespace is a system (implemented as a dictionary) that ensures all names in a program are unique and can be used without conflict.

- **Built-in:** Contains all integrated symbols available in Python (e.g., `print()`).
- **Global:** Symbols defined at the top level of a module or script.
- **Local:** Created for specific scopes, such as within a function.
- **LEGB Rule:** Python searches for names from the inside out: **L**ocal → **G**lobal → **B**uilt-in.

Built-in symbols can be overwritten by the user, however, it is not recommended.

4 Arithmetic and Magic Methods

- **Special Operators:** `//` (integer division), `%` (modulo/remainder), and `**` (exponentiation).
- **Syntactic Sugar:** Operators are just a "friendly"way to call internal methods. For example, `x + y` is internally executed as `x.__add__(y)`.

Operator	Dunder Method	Description
`x + y`	`x.__add__(y)`	Sum
`x - y`	`x.__sub__(y)`	Difference
`x * y`	`x.__mul__(y)`	Product
`x / y`	`x.__truediv__(y)`	Quotient
`x // y`	`x.__floordiv__(y)`	Floored quotient[1]
`x % y`	`x.__mod__(y)`	Remainder (Modulo)[1]
`x ** y`	`x.__pow__(y)`	Exponentiation (x^y)
`abs(x)`	`x.__abs__()`	Absolute value
`-x`	`x.__neg__()`	Negative sign

[1] Operator not defined for the data type `complex`

Code Example

Ecco come Python interpreta gli operatori dietro le quinte:

```python
# Standard operator usage
result_a = 5 + 5

# Behind the scenes (Magic Method)
result_b = (5).__add__(5)

print(result_a == result_b) # Output: True
```

5 Conditional Branching

- **Basic Structures**: `if`, `elif`, and `else` execute code blocks based on conditions. Consistent indentation is required.
- **Match-case**: Introduced in Python 3.10 to recognize specific patterns and data structures.

```python
x = 5

if x == 5:
    print("5")
else:
    print("Not 5")

match x:
    case 5:
        print("5")
    case 4:
        print("4")
    case _:
        print("Default case")
```

5.1 Boolean Expressions and Operators

- **Walrus Operator (`:=`)**: Assigns and returns a value within an expression.
- **Ternary Operator**: Compact syntax for simple single-line conditional assignments.
- **`==` vs `is`**:
  - `==` checks value equality;
  - `is` checks object identity in memory (ideal for checking `None`).
- **`in` Operator**: Checks presence in objects implementing `__contains__()` (Only usable on iterable Objects: lists, strings, dicts).

```python
# Walrus Operator (:=)
if (a := int(input("Enter a number: "))) > 3:
    print("Number is greater than 3.")

# Ternary Operator
b = 5
a = b if b > 3 else 0

# == vs is
my_list = [1, 2, 3]
print(my_list == [1, 2, 3])  # True (values are identical)
print(my_list is [1, 2, 3])  # False (different objects in memory)

# in Operator
fruits = ["banana", "apple"]
print("apple" in fruits)     # True
```

5.2 Exception Handling

- **LBYL (Look Before You Leap)**: A defensive style where you explicitly check for conditions (`if os.path.exists`) before making a call. This can be prone to "race conditions"where the state changes between the check and the action.
- **EAFP (Easier to Ask Forgiveness than Permission)**: The 'Pythonic' approach. Assume the operation will succeed and wrap it in a `try/except` block. This is generally cleaner and more robust in multi-threaded environments.
- **Catching Exceptions**: Always catch specific exceptions (e.g., `FileNotFoundError`) rather than generic ones. Use the `as` keyword to inspect the exception object.
- **Flow Control**:
  - `else`: Runs only if the `try` block succeeded without errors.
  - `finally`: Always runs (e.g., for closing file handles), regardless of whether an exception was raised or caught.

```python
# EAFP: try, except, else, finally
try:
    x = int(input("Enter a number other than zero: "))
    y = 1 / x
except ValueError:
    print('Oops. Not a valid integer.')
except ZeroDivisionError as e:
```

```
    print('Oops! ' + str(e))
else:
    print(f"Success! Result: {y}")
finally:
    print("Always executed.")

# LBYL
inp = input("Enter a number other than zero: ")

if(inp.isdecimal()):
    x = int(inp)
    if x != 0:
        print(1/x)
    else:
        print("Wrong Input!")
else:
    print("Wrong Input!")
```

6 Iterations

6.1 Slicing

- Syntax: `part = x[start:end:step]`.
- Start is inclusive (default 0).
- End is exclusive (default end index).
- Step is optional (default 1). A negative step indicates reverse order.

```
data = [0, 1, 2, 3, 4, 5.0, [1,2,3]]
part = data[::-1] # Reverse order
```

6.2 For-loops

- Internally calls `iter()` to return an iterator.
- Uses the `next(iterator)` method to get the next element until a `StopIteration` exception is raised.

```
for value in iterable:
    expression1
```

6.2.1 Loop-control statements

- **break**: Terminates a loop before completion.
- **continue**: Skips the remaining code in the loop for the current iteration.
- **else keyword**: Executed when the loop ends normally, meaning it has not encountered a `break` statement.

6.2.2 Dictionaries

```
for key in data:                # Iterates keys
for value in data.values():     # Iterates values
for key, value in data.items(): # Iterates both keys and values
```

6.2.3 Enumerate

- Used when a counter is needed while iterating through elements.

```
for i, fruit in enumerate(fruits, start=10):
    print(i, fruit)
```

6.2.4 Zip

- Iterates over multiple iterables at the same time.
- **Objects of different lengths**: The `zip()` function stops as soon as the shortest iterable is exhausted. Any remaining elements in the longer iterables are simply ignored.

```
colors = ["yellow", "red", "yellow", "blue", "green"] # Length 5
fruits = ["banana", "apple", "lemon", "blueberry"]    # Length 4

# The loop will only run 4 times
for color, fruit in zip(colors, fruits):
    print(color, fruit)
```

6.3 Comprehensions

- **Base Syntax**: `[expression for item in iterable]`.
- **Nested for loops**: Evaluated from left to right.
- **Can also call functions**: Built-in functions, like `any()`, can be used.
- **Conditional Syntax**: The position of the `if` statement completely changes its behavior.
  - **Filtering (Only `if`)**: Must be placed at the **END**. It filters elements out of the iterable.
  - **Modifying (`if/else`)**: Must be placed at the **BEGINNING** (before the `for` loop). It changes the value of the element based on the condition but does not filter it out.

```
# 1. Nested loops
# Syntax: [expression for outer_item in outer_iterable for inner_item in inner_iterable]
fields = [b + str(n) for b in letters for n in numbers]

# Normal form equivalent
for b in letters:
    for n in numbers:
        b + str(n)

# 2. ONLY IF (Filtering) -> Placed at the END
# Syntax: [expression for item in iterable if condition]
filtered_values = [item**2 for item in range(1, 11) if (item**2) % 4 != 0]

# 3. IF/ELSE (Modifying) -> Placed at the BEGINNING
# Syntax: [expression_if_true if condition else expression_if_false for item in iterable]
modified_values = [item**2 if (item**2) % 4 != 0 else 0 for item in range(1, 11)]
```